

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №7**

**«Робота з базами даних: SQLite, DB-API, SQLAlchemy ORM, Repository,
міграції»**

з курсу «Професійний Python»
Варіант №1 — «Система обліку студентів»

Виконав: Лис Артур

Група: ФЕП-31

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Побудувати persistence layer для проєкту обліку студентів: спроектувати дві пов'язані таблиці в SQLite, виконати CRUD через стандартний DB-API з параметризованими запитамі, перенести ту саму схему на SQLAlchemy ORM за патерном Repository, реалізувати транзакційні операції з commit/rollback, керувати схемою через міграції Alembic і перевірити все інтеграційними тестами на окремій тестовій базі.

2. Індивідуальний варіант і постановка задачі

Варіант 1 — система обліку студентів. Зберігати ім'я, прізвище, групу та середній бал; таблиці groups і students; операції: додавання, виведення, пошук за прізвищем, фільтр за групою, зміна балу, переведення до іншої групи, видалення, найкращий студент, середній бал групи; репозиторії StudentRepository та GroupRepository; міграція — додати поле email; інтеграційні тести на Create, Read, Update, Delete та filter_by_group.

Понад мінімум: generic SqlRepository[T], Unit of Work у вигляді session_scope, CHECK-і UNIQUE-обмеження, увімкнене enforcement зовнішніх ключів у SQLite, каскадне видалення, обробка IntegrityError, міграція з upgrade та downgrade, JOIN + GROUP BY з COUNT/AVG, сортування й пагінація, тест rollback, тест обмежень, тест міграцій, а також власна SQL-функція casefold для пошуку кирилицею без урахування регістру.

3. Предметна область і схема бази даних

Одна група має багато студентів (one-to-many). Група ідентифікується кодом на кшталт «ФЕП-31с»; студент посилається на групу через group_id. Видалення групи каскадно видаляє її студентів — так було вирішено, бо студент без групи в цій моделі не існує.

groups		students
-----		-----
id INTEGER PK	1 ----- *	id INTEGER PK
code TEXT UNIQUE		first_name TEXT NOT NULL
		last_name TEXT NOT NULL (index)
		average_grade REAL NOT NULL CHECK 0..100
		group_id INTEGER NOT NULL FK -> groups.id ON DELETE CASCADE
		email TEXT NULL UNIQUE (міграція 0002)

Таблиця	Поле	Тип	Обмеження
groups	id	INTEGER	PRIMARY KEY
groups	code	VARCHAR(16)	NOT NULL, UNIQUE
students	id	INTEGER	PRIMARY KEY

students	first_name / last_name	VARCHAR(64)	NOT NULL; індекс за last_name
students	average_grade	FLOAT	NOT NULL, CHECK (0 ≤ x ≤ 100)
students	group_id	INTEGER	NOT NULL, FOREIGN KEY → groups.id ON DELETE CASCADE
students	email	VARCHAR(120)	NULL, UNIQUE — додано міграцією 0002

3.1. SQL створення таблиць і стан бази після демонстрації

```
$ sqlite3 data/students.db ".schema"
CREATE TABLE alembic_version (
    version_num VARCHAR(32) NOT NULL,
    CONSTRAINT alembic_version_pkc PRIMARY KEY (version_num)
);
CREATE TABLE groups (
    id INTEGER NOT NULL,
    code VARCHAR(16) NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uq_groups_code UNIQUE (code)
);
CREATE TABLE "students" (
    id INTEGER NOT NULL,
    first_name VARCHAR(64) NOT NULL,
    last_name VARCHAR(64) NOT NULL,
    average_grade FLOAT NOT NULL,
    group_id INTEGER NOT NULL,
    email VARCHAR(120),
    PRIMARY KEY (id),
    CONSTRAINT ck_students_grade_range CHECK (average_grade >= 0 AND average_grade <= 100),
    CONSTRAINT uq_students_email UNIQUE (email),
    FOREIGN KEY(group_id) REFERENCES groups (id) ON DELETE CASCADE
);
CREATE INDEX ix_students_group_id ON students (group_id);
CREATE INDEX ix_students_last_name ON students (last_name);

PRAGMA foreign_key_list(students): [(0, 0, 'groups', 'group_id', 'id', 'NO ACTION', 'CASCADE', 'NONE')]

SELECT s.id, s.last_name, g.code, s.average_grade, s.email FROM students s JOIN groups g ON g.id =
s.group_id;
(1, 'Гнатишин', 'ФЕП-31с', 93.4, None)
(2, 'Дзюба', 'ФЕП-31с', 81.5, None)

SELECT * FROM alembic_version; [('0002',)]
```

4. Структура проєкту

```
|— alembic.ini
|— migrations
|   |— env.py
```

```

├── script.py.mako
├── versions
│   ├── 0001_initial_schema.py
│   └── 0002_add_student_email.py
├── db
│   ├── __init__.py
│   ├── __main__.py
│   ├── engine.py
│   ├── exceptions.py
│   ├── orm.py
│   ├── raw.py
│   ├── repositories.py
│   └── services.py
└── tests
    ├── conftest.py
    ├── integration
    │   ├── conftest.py
    │   ├── test_container.py
    │   ├── test_csv_roundtrip.py
    │   ├── test_db.py
    │   └── test_pipeline.py
    └── unit
        ├── test_analytics.py
        ├── test_config.py
        ├── test_domain.py
        ├── test_models.py
        ├── test_notify.py
        ├── test_registry.py
        ├── test_snapshot.py
        ├── test_storage.py
        └── test_streaming.py

```

student_manager/db — весь persistence layer: orm.py (моделі), engine.py (engine, session factory, session_scope), repositories.py, services.py (бізнес-операції = транзакції), raw.py (DB-API без ORM), exceptions.py, демонстрація в __main__.py. migrations/ + alembic.ini — Alembic. Тести БД лежать у tests/integration/test_db.py з власним conftest.py.

5. DB-API: sqlite3, параметризовані запити, commit і rollback

Модуль raw.py працює з тими самими таблицями напряму через sqlite3: Connection тримає транзакцію, Cursor виконує запит. Значення передаються окремо від тексту SQL через ? — драйвер підставляє їх як дані, тож користувацький рядок не може змінити структуру запиту.

```

def connect(path: Path | str) -> sqlite3.Connection:
    connection = sqlite3.connect(path)
    connection.row_factory = sqlite3.Row # rows behave like dicts: row["last_name"]
    connection.execute("PRAGMA foreign_keys = ON")
    return connection

def insert_student(connection: sqlite3.Connection, first: str, last: str, group_id: int, grade: float)
-> int:
    cursor = connection.execute(
        "INSERT INTO students (first_name, last_name, group_id, average_grade) VALUES (?, ?, ?, ?)",
        (first, last, group_id, grade),
    )

```

```

    )
    return int(cursor.lastrowid or 0)

def find_by_last_name(connection: sqlite3.Connection, last_name: str) -> list[sqlite3.Row]:
    """Parameterised: the value travels separately from the SQL text, so it can never become SQL."""
    cursor = connection.execute(
        "SELECT s.id, s.first_name, s.last_name, g.code, s.average_grade "
        "FROM students s JOIN groups g ON g.id = s.group_id "
        "WHERE s.last_name = ? ORDER BY s.last_name, s.first_name",
        (last_name,),
    )
    return list(cursor.fetchall())

def find_by_last_name_unsafe(connection: sqlite3.Connection, last_name: str) -> list[sqlite3.Row]:
    """Anti-example for the report: string formatting lets the input rewrite the query."""
    query = f"SELECT id, last_name FROM students WHERE last_name = '{last_name}'"
    return list(connection.execute(query).fetchall())

def transfer(connection: sqlite3.Connection, student_id: int, group_code: str) -> None:
    """Transactional move: the UPDATE is committed only if the target group exists; otherwise rolled back."""
    try:
        row = connection.execute("SELECT id FROM groups WHERE code = ?", (group_code,)).fetchone()
        if row is None:
            raise LookupError(f"групи {group_code} не існує")
        connection.execute("UPDATE students SET group_id = ? WHERE id = ?", (row["id"], student_id))
        connection.commit()
    except Exception:
        connection.rollback()
        raise

```

Результат: на payload ' OR '1'='1 параметризований запит повертає 0 рядків, а f-string — усю таблицю. У transfer commit() виконується лише після успішного UPDATE; будь-яка помилка веде до rollback(), і group_id студента лишається незмінним.

DB-API (sqlite3): CREATE TABLE, INSERT ?, SELECT ?, JOIN + GROUP BY

```

-----
rows: [{ 'id': 1, 'first_name': 'Марта', 'last_name': 'Гнатишин', 'code': 'ФЕП-31c', 'average_grade': 93.4}]
group averages: [('ФЕП-31c', 2, 86.15), ('ФЕП-32', 2, 76.05)]

```

Parameterized query проти f-string: payload = "' OR '1'='1"

```

-----
safe    -> 0 рядків
unsafe  -> 4 рядків (весь список витік)

```

Транзакція з rollback (DB-API)

```

-----
transfer(1, 'ФЕП-99'): LookupError: групи ФЕП-99 не існує
group_id студента 1 до/після: 1 / 1 (UPDATE не відбувся)
transfer(1, 'ФЕП-32') -> group_id = 2

```

6. database.py: engine, session factory, session_scope

SQLite за замовчуванням не перевіряє зовнішні ключі — це вмикається `PRAGMA foreign_keys = ON` на кожному з'єднанні, тому хук `connect` робить це автоматично і заодно реєструє SQL-функцію `casefold`. `session_scope` — Unit of Work: одна сесія, один `commit()` у кінці блоку або `rollback()` при винятку.

```
def make_engine(url: str = DEFAULT_URL, *, echo: bool = False) -> Engine:
    """SQLite ignores FOREIGN KEY unless every connection turns enforcement on; the connect hook does
    that."""
    engine = create_engine(url, echo=echo)

    @event.listens_for(engine, "connect")
    def configure_connection(dbapi_connection: Any, _record: Any) -> None:
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA foreign_keys = ON")
        cursor.close()
        # SQLite's LIKE and lower() are case-insensitive for ASCII only; register Python's casefold as
        an
        # SQL function so that a search for "гн" finds "Гнатишин".
        dbapi_connection.create_function("casefold", 1, _casefold, deterministic=True)

    return engine

@contextmanager
def session_scope(factory: sessionmaker[Session]) -> Iterator[Session]:
    """One transaction per ``with`` block: commit on success, rollback on any exception, close
    always."""
    session = factory()
    try:
        yield session
        session.commit()
    except BaseException:
        session.rollback()
        raise
    finally:
        session.close()
```

7. ORM-моделі SQLAlchemy

```
class Base(DeclarativeBase):
    """Declarative base; ``Base.metadata`` knows every table below (used by create_all and Alembic)."""

class GroupRecord(Base):
    __tablename__ = "groups"

    id: Mapped[int] = mapped_column(primary_key=True)
    code: Mapped[str] = mapped_column(String(16), unique=True)

    # one-to-many; deleting a group deletes its students on the database side (ON DELETE CASCADE),
    # passive_deletes tells the ORM not to load them first just to delete them one by one.
    students: Mapped[list[StudentRecord]] = relationship(
        back_populates="group", cascade="all, delete-orphan", passive_deletes=True,
```

```

        order_by="StudentRecord.last_name",
    )

class StudentRecord(Base):
    __tablename__ = "students"
    __table_args__ = (
        CheckConstraint(f"average_grade >= {GRADE_MIN:g} AND average_grade <= {GRADE_MAX:g}",
                        name="ck_students_grade_range"),
        UniqueConstraint("email", name="uq_students_email"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    first_name: Mapped[str] = mapped_column(String(64))
    last_name: Mapped[str] = mapped_column(String(64), index=True)
    average_grade: Mapped[float]
    group_id: Mapped[int] = mapped_column(ForeignKey("groups.id", ondelete="CASCADE"), index=True)
    email: Mapped[str | None] = mapped_column(String(120)) # migration 0002

    group: Mapped[GroupRecord] = relationship(back_populates="students")

    @property
    def full_name(self) -> str:
        return f"{self.last_name} {self.first_name}"

    def to_domain(self) -> Student:
        """The lab-1 dataclass, so the registry, analytics and reports keep working unchanged."""
        return Student(self.first_name, self.last_name, self.group.code, self.average_grade)

```

`relationship()` з `back_populates` дає навігацію в обидва боки: `group.students` і `student.group`. `to_domain()` повертає dataclass `Student` з ЛР1, тому реєстр, аналітика й звіти попередніх робіт працюють поверх бази без змін.

8. Repository Pattern

`SqlRepository[T]` інкапсулює загальний CRUD над будь-якою моделлю; конкретні репозиторії додають запити варіанта. Репозиторій ніколи не викликає `commit()` — межі транзакції визначає сервіс. `flush()` після INSERT/UPDATE потрібен, щоб обмеження бази спрацювали одразу, а не десь наприкінці транзакції.

```

class SqlRepository(Generic[T]):
    def __init__(self, session: Session, model: type[T]) -> None:
        self._session = session
        self._model = model

    def add(self, item: T) -> T:
        self._session.add(item)
        self._session.flush() # INSERT now, so constraint errors surface here and the id is assigned
        return item

    def get(self, item_id: int) -> T | None:
        return self._session.get(self._model, item_id)

    def require(self, item_id: int) -> T:

```

```

        item = self.get(item_id)
        if item is None:
            raise RecordNotFoundError(str(self._model.__tablename__), item_id)
        return item

    def all(self, *order_by: ColumnExpressionArgument[Any], limit: int | None = None, offset: int = 0)
-> list[T]:
        statement = select(self._model).order_by(*order_by).offset(offset).limit(limit)
        return list(self._session.scalars(statement).all())

    def by_last_name(self, last_name: str) -> list[StudentRecord]:
        prefix = last_name.strip().casefold()
        statement = (select(StudentRecord)
            .where(func.casefold(StudentRecord.last_name).like(f"{prefix}%")) # casefold: see
engine.py
            .order_by(StudentRecord.last_name, StudentRecord.first_name))
        return list(self._session.scalars(statement).all())

    def filter_by_group(self, code: str) -> list[StudentRecord]:
        statement = (select(StudentRecord).join(StudentRecord.group)
            .where(GroupRecord.code == code.strip())
            .order_by(StudentRecord.last_name))
        return list(self._session.scalars(statement).all())

    def best(self, code: str | None = None) -> StudentRecord | None:
        statement = select(StudentRecord).order_by(StudentRecord.average_grade.desc(),
StudentRecord.id).limit(1)
        if code is not None:
            statement = statement.join(StudentRecord.group).where(GroupRecord.code == code)
        return self._session.scalar(statement)

    def statistics(self) -> list[GroupStats]:
        """JOIN + GROUP BY + COUNT/AVG in one round trip; groups without students give average None."""
        statement = (
            select(GroupRecord.code, func.count(StudentRecord.id),
func.avg(StudentRecord.average_grade))
            .outerjoin(StudentRecord, StudentRecord.group_id == GroupRecord.id)
            .group_by(GroupRecord.id)
            .order_by(GroupRecord.code)
        )
        return [GroupStats(code, students, average) for code, students, average in
self._session.execute(statement)]

```

9. Сервіс, транзакції та ACID

Кожен публічний метод `StudentService` — одна транзакція через `_transaction()`, який обгортає `session_scope` і перетворює `IntegrityError` SQLAlchemy на власний `ConstraintViolationError`, щоб решта проєкту не залежала від ORM.

```

def import_many(self, rows: Iterable[NewStudent]) -> int:
    """All or nothing: one bad row rolls back every row inserted before it (atomicity)."""
    added = 0
    with self._transaction(on_row=lambda: added + 1) as session:
        for row in rows:
            group = GroupRepository(session).get_or_create(row.group)
            StudentRepository(session).add(StudentRecord(

```

```

        first_name=row.first_name, last_name=row.last_name,
        average_grade=row.average_grade,
        email=row.email, group=group,
    ))
    added += 1
    return added

def transfer(self, student_id: int, group_code: str) -> Student:
    """Move to another group, creating it if needed: two writes that must succeed or fail
    together."""
    with self._transaction() as session:
        student = StudentRepository(session).require(student_id)
        student.group = GroupRepository(session).get_or_create(group_code)
        session.flush()
        return student.to_domain()

@contextmanager
def _transaction(self, on_row: Callable[[], int] | None = None) -> Iterator[Session]:
    """session_scope plus translation of IntegrityError into the package's own exception.
    The rollback has already happened (inside session_scope) by the time the error is re-raised."""
    try:
        with session_scope(self._factory) as session:
            yield session
    except IntegrityError as error:
        where = f" (рядок {on_row()})" if on_row is not None else ""
        raise ConstraintViolationError(f"база даних відхилила зміну{where}: {error.orig}") from
error

```

Властивість	Як забезпечено в проєкті
Atomicity	`import_many`: три INSERT в одній сесії; третій рядок порушує CHECK — `session_scope` викликає `rollback()`, і жоден із попередніх рядків (і нова група) не зберігається.
Consistency	CHECK на діапазон балу, UNIQUE на код групи та email, FOREIGN KEY з увімкненим enforcement: база не приймає стан, що порушує правила, незалежно від помилок у Python-кодi.
Isolation	SQLite блокує файл на час запису; кожен виклик сервісу — окрема сесія, яка бачить лише зафіксовані дані інших транзакцій.
Durability	Після `commit()` зміни записані у файл `students.db` через журнал; демонстрація читає той самий файл окремим з'єднанням `sqlite3` уже після закриття сесії.

9.1. Обмеження та rollback у дії

Обмеження та rollback (ORM)

```

set_grade(1, 150): ConstraintViolationError: база даних відхилила зміну: CHECK constraint failed:
ck_students_grade_range
add(duplicate email): ConstraintViolationError: база даних відхилила зміну: UNIQUE constraint failed:
students.email
transfer(99, ...): RecordNotFoundError: у таблиці students немає запису з id=99
import_many([ok, ok, grade=-5]): ConstraintViolationError: база даних відхилила зміну (рядок 3):
CHECK constraint failed: ck_students_grade_range

```

студентів до/після імпорту: 4 / 4 – жоден рядок не зберігся
оцінка студента 1 після невдалого set_grade: 93.4

10. Міграції Alembic

Схема реальної бази створюється не create_all, а міграціями: 0001 буде таблиці, 0002 додає поле email з UNIQUE-обмеженням. SQLite не вміє змінювати обмеження на місці, тому batch_alter_table перебудовує таблицю; downgrade повертає схему назад.

```
[alembic]
script_location = migrations
prepend_sys_path = src
sqlalchemy.url = sqlite:///data/students.db
file_template = %(rev)s_%(slug)s

def run_migrations_online() -> None:
    engine = create_engine(url, poolclass=pool.NullPool)
    with engine.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata, render_as_batch=True)
        with context.begin_transaction():
            context.run_migrations()

def upgrade() -> None:
    # SQLite cannot ALTER a constraint in place; batch mode rebuilds the table with the new definition.
    with op.batch_alter_table("students") as batch:
        batch.add_column(sa.Column("email", sa.String(length=120), nullable=True))
        batch.create_unique_constraint("uq_students_email", ["email"])

def downgrade() -> None:
    with op.batch_alter_table("students") as batch:
        batch.drop_constraint("uq_students_email", type_="unique")
        batch.drop_column("email")

$ alembic history
0001 -> 0002 (head), add email to students (variant 1 migration)
<base> -> 0001, initial schema: groups and students

$ alembic current
0002 (head)

Alembic: upgrade 0001 -> head
-----
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 0001, initial schema: groups and students
students після 0001: ['id', 'first_name', 'last_name', 'average_grade', 'group_id']
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade 0001 -> 0002, add email to students (variant 1
migration)
students після head: ['id', 'first_name', 'last_name', 'average_grade', 'group_id', 'email']
таблиці: ['alembic_version', 'groups', 'students']

Alembic: downgrade -1 і назад
-----
```

```

INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running downgrade 0002 -> 0001, add email to students (variant 1
migration)
students після downgrade: ['id', 'first_name', 'last_name', 'average_grade', 'group_id']
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade 0001 -> 0002, add email to students (variant 1
migration)
students після upgrade:  ['id', 'first_name', 'last_name', 'average_grade', 'group_id', 'email']

```

11. Інтеграційні тести на окремій базі

Фікстура engine створює новий файл SQLite у tmp_path для кожного тесту і закриває пул у teardown; service наповнює базу трьома студентами. Реальна data/students.db тестами не торкається.

```

@pytest.fixture
def engine(db_path: Path) -> Iterator[Engine]:
    engine = make_engine(f"sqlite:/// {db_path.as_posix()}")
    create_schema(engine)
    yield engine
    engine.dispose() # teardown: close the pool so tmp_path can be deleted on Windows

@pytest.fixture
def factory(engine: Engine) -> sessionmaker[Session]:
    return make_session_factory(engine)

@pytest.fixture
def service(factory: sessionmaker[Session]) -> StudentService:
    service = StudentService(factory)
    for row in SEED:
        service.add(row)
    return service

def test_create_and_read(service: StudentService) -> None:
    new_id = service.add(NewStudent("Ярина", "Бойко", "ФЕП-33", 71.0))
    listing = service.listing()
    assert new_id == 4 and len(listing) == 4
    assert listing[-1].full_name == "Бойко Ярина" and listing[-1].group == "ФЕП-33"

def test_filter_by_group(service: StudentService) -> None:
    assert [s.last_name for s in service.in_group("ФЕП-31с")] == ["Гнатишин", "Дзюба"]
    assert service.in_group("ФЕП-99") == []

def test_delete(service: StudentService) -> None:
    assert service.remove(1) is True
    assert service.remove(1) is False
    assert [s.last_name for s in service.listing()] == ["Дзюба", "Кравець"]

def test_check_constraint_rejects_grade_above_100(service: StudentService) -> None:
    with pytest.raises(ConstraintViolationError, match="ck_students_grade_range|CHECK"):
        service.set_grade(1, 150)

```

```

assert service.listing()[0].average_grade == 93.4 # the UPDATE was rolled back

def test_import_many_is_atomic(service: StudentService) -> None:
    rows = [NewStudent("Ярина", "Бойко", "ФЕП-33", 90), NewStudent("Роман", "Скиба", "ФЕП-33", 85),
            NewStudent("Олег", "Шевчук", "ФЕП-33", -5)]
    with pytest.raises(ConstraintViolationError, match="рядок 3"):
        service.import_many(rows)
    assert len(service.listing()) == 3
    assert service.in_group("ФЕП-33") == []
    assert all(g.code != "ФЕП-33" for g in service.group_statistics()) # the new group was rolled back
    too

def test_migrations_upgrade_and_downgrade(tmp_path: Path) -> None:
    db = tmp_path / "migrated.db"
    config = Config()
    config.set_main_option("script_location", str(ROOT / "migrations"))
    config.set_main_option("sqlalchemy.url", f"sqlite:///{"db.as_posix()}")

    command.upgrade(config, "0001")
    assert columns(db, "students") == ["id", "first_name", "last_name", "average_grade", "group_id"]
    command.upgrade(config, "head")
    assert columns(db, "students")[-1] == "email"
    command.downgrade(config, "-1")
    assert "email" not in columns(db, "students")
    command.upgrade(config, "head")
    assert "email" in columns(db, "students")

```

11.1. pytest -v

```

$ pytest tests/integration/test_db.py -v
tests/integration/test_db.py::test_create_and_read PASSED
tests/integration/test_db.py::test_read_by_last_name_prefix_is_case_insensitive PASSED
tests/integration/test_db.py::test_filter_by_group PASSED
tests/integration/test_db.py::test_update_grade PASSED
tests/integration/test_db.py::test_update_transfer_creates_the_target_group PASSED
tests/integration/test_db.py::test_delete PASSED
tests/integration/test_db.py::test_best_and_group_average_use_aggregates PASSED
tests/integration/test_db.py::test_pagination_and_sorting PASSED
tests/integration/test_db.py::test_check_constraint_rejects_grade_above_100 PASSED
tests/integration/test_db.py::test_unique_email_is_enforced PASSED
tests/integration/test_db.py::test_foreign_key_is_enforced PASSED
tests/integration/test_db.py::test_import_many_is_atomic PASSED
tests/integration/test_db.py::test_rollback_on_python_error_inside_the_scope PASSED
tests/integration/test_db.py::test_missing_record_raises PASSED
tests/integration/test_db.py::test_deleting_a_group_cascades_to_students PASSED
tests/integration/test_db.py::test_repository_count_and_get_or_create PASSED
tests/integration/test_db.py::test_dbapi_parameterized_query_is_injection_safe PASSED
tests/integration/test_db.py::test_migrations_upgrade_and_downgrade PASSED
===== 18 passed in 1.38s =====

$ pytest -q
..... [100%]
193 passed in 3.17s

```

12. Результати роботи програми

```

Create: service.add / import_many
-----

```

1 Гнатишин Марта	ФЕП-31с	93.40
2 Дзюба Остап	ФЕП-31с	78.90
3 Кравець Соломія	ФЕП-32	88.10
4 Гаврилюк Тарас	ФЕП-32	64.00

Read: пошук за прізвищем, фільтр за групою, найкращий, середній бал групи

```
-----
find('Гн')      -> ['Гнатишин Марта']
in_group(ФЕП-32)-> ['Гаврилюк Тарас', 'Кравець Соломія']
best()          -> Гнатишин Марта 93.4
group_average(ФЕП-31с) -> 86.15
statistics      -> [('ФЕП-31с', 2, 86.15), ('ФЕП-32', 2, 76.05)]
```

Update: зміна балу, переведення до іншої групи

```
-----
set_grade(2, 81.5) -> Student(first_name='Остап', last_name='Дзюба', group='ФЕП-31с',
average_grade=81.5)
transfer(4, 'ФЕП-41') -> Student(first_name='Тарас', last_name='Гаврилюк', group='ФЕП-41',
average_grade=64.0)
statistics -> [('ФЕП-31с', 2), ('ФЕП-32', 1), ('ФЕП-41', 1)]
```

Delete: студент, каскадне видалення групи

```
-----
remove(3) -> True ; remove(3) вдруге -> False
remove_group('ФЕП-41') -> видалено студентів: 1
1 Гнатишин Марта      ФЕП-31с    93.40
2 Дзюба Остап        ФЕП-31с    81.50
```

13. Перевірка типів і Git

```
$ mypy src
Success: no issues found in 64 source files
```

```
$ git log --oneline -4
520df89 lab7: integration tests on a per-test SQLite file, README section, version 0.7.0
8c933f0 lab7: Alembic setup with initial schema and email migration (upgrade/downgrade)
89c43f6 lab7: db package – ORM models, engine/session_scope, repositories, transactional service, DB-API module
fe1755b lab6: pytest markers, branch coverage with 80% threshold, README section, version 0.6.0
```

```
$ git status --short
(чисто)
```

14. Відповіді на контрольні запитання

1. Що таке реляційна база даних?

Дані у таблицях із рядками та полями, пов'язаних ключами; запити — мовою SQL.

2. Що означає CRUD і які SQL-команди йому відповідають?

Create → INSERT, Read → SELECT, Update → UPDATE, Delete → DELETE.

3. Які переваги SQLite?

Один файл без сервера, входить у стандартну бібліотеку Python, транзакційна, достатня для локальних застосунків і тестів.

4. Для чого Connection і Cursor у DB-API?

Connection тримає з'єднання і транзакцію (commit/rollback); Cursor виконує запити та віддає рядки.

5. Для чого commit() і rollback()?

commit фіксує зміни транзакції у файлі; rollback скасовує все, зроблене після останнього commit.

6. Що таке SQL injection і чому не варто формувати SQL через f-string?

Вхідний рядок стає частиною SQL і змінює запит (' OR '1'='1 повернув усю таблицю); параметри ? передаються окремо як дані.

7. Що таке Primary Key і Foreign Key?

PK унікально ідентифікує рядок; FK — поле, що посилається на PK іншої таблиці і підтримує цілісність зв'язку.

8. Що таке транзакція та ACID?

Група операцій, що виконується як одне ціле: Atomicity (усе або нічого), Consistency (обмеження не порушуються), Isolation (транзакції не бачать незафіксованих змін), Durability (після commit дані збережено).

9. Що таке persistence layer і навіщо Repository Pattern?

Шар, що ховає SQL/ORM за інтерфейсом «колекції» об'єктів; бізнес-логіка не знає, звідки беруться дані, і її можна тестувати з in-memory заміною.

10. Чому Repository відокремлюють від Service Layer?

Репозиторій відповідає за доступ до даних, сервіс — за правила й межі транзакцій; інакше commit розповзається по всьому коду.

11. Що таке ORM і що таке ORM model?

Object-Relational Mapping — відображення таблиць на класи; модель — клас із Mapped-полями, що описує таблицю та зв'язки.

12. Що таке relationship() і one-to-many?

Атрибут, через який ORM завантажує пов'язані об'єкти; one-to-many — один рядок батьківської таблиці має багато дочірніх (група → студенти).

13. Що таке many-to-many?

Зв'язок через проміжну таблицю з двома FK (наприклад, студенти ↔ курси).

14. Що таке migration і для чого Alembic та alembic upgrade head?

Версійована зміна схеми; Alembic зберігає ланцюжок ревізій у alembic_version, upgrade head доводить базу до останньої.

15. Навіщо окрема test database?

Тести змінюють і видаляють дані; свіжий файл у tmp_path робить їх незалежними і не псує робочу базу.

16. Що перевіряє тест rollback?

Що після помилки всередині транзакції в базі немає жодних її часткових змін (бал лишився 93.4, група ФЕП-33 не з'явилася).

17. Що робить JOIN і GROUP BY?

JOIN поєднує рядки двох таблиць за умовою (студент + його група); GROUP BY згортає рядки в групи для агрегатів COUNT/AVG.

18. Для чого COUNT, SUM, AVG, MIN, MAX?

Агрегатні функції над групою рядків: кількість, сума, середнє, мінімум, максимум — рахуються в базі, а не в Python.

15. Висновок

До проєкту додано persistence layer на SQLite: дві пов'язані таблиці з PK, FK, CHECK і UNIQUE, CRUD і агрегати через DB-API з параметризованими запитамі та через SQLAlchemy ORM за патерном Repository, транзакційний сервіс із commit/rollback і власними винятками, дві міграції Alembic з upgrade/downgrade. 18 інтеграційних тестів на окремій базі перевіряють CRUD, filter_by_group, обмеження, атомарність імпорту, rollback і міграції; разом у проєкті 193 тести, туру у strict-режимі чистий. Практичний висновок: SQLite мовчки не перевіряє зовнішні ключі та не знає кирилиці в LIKE — обидва питання вирішено на рівні з'єднання, а не обходами в запитах.